

## Chapter 11. Implementing Observability in Java

In this chapter, you will see how to apply the principles and ideas of the last chapter to real, production Java/JVM systems. This will include the three pillars of metrics, traces, and logs.

We will use three primary technologies for our examples—Micrometer, Prometheus, and OpenTelemetry. However, it should be clearly understood that these technologies have different domains of applicability, and in practice, many real systems will use some or all of them together to provide a complete observability implementation.

There are also many other technologies in use in the field, with varying levels of maturity—in fact, one of the difficult problems in observability is managing the complexity of possible deployments.

A second, and related problem: observability is, by design, intended to be used to understand complex software systems and varied architectures. This means that, while there is an emerging set of patterns, there is no single “right” way to implement observability—the best solution for a particular software system depends on the details.

In this chapter, we'll be using Fighting Animals, from [Chapter 8](#), as our example application.

---

### WARNING

The architectural choices we make for observing Fighting Animals are not necessarily the best choices for other applications—a lot depends upon the details of the application architecture and the environment in which it is deployed.

---

Let's start looking at the Micrometer library to see how it can be applied to our example application.

## Introducing Micrometer

At the time of writing (August 2024), one of the most common—and also most effective—of the current Java metrics libraries is [Micrometer](#), for-

merly developed as part of the Spring project but now standalone. It is a Java/JVM project, so teams that want to have the same library used across a heterogeneous architecture will need to consider alternatives.

The project is probably best described as a vendor-neutral application metrics facade, although it has other subprojects that also handle other observability data types, such as tracing.

Micrometer integrates with a large number of metrics data sources and backends, including: Azure Monitor, CloudWatch, Datadog, Dynatrace, Elastic, JMX, New Relic, OpenTelemetry Protocol (OTLP), Prometheus, SignalFx, and StatsD.

It is a library intended for use by the application developer, and as such, it expects the developer to consciously create and record the metrics they want to collect.

## Meters and Registries

Micrometer provides a core library as a service provider interface (SPI) and uses pluggable *metric consumers*, which are service implementations that export to various vendor and OSS solutions. This is similar to how a logging framework operates (i.e., both are examples of the facade pattern). This structure enables the framework to handle any differences between metrics systems by delegating any required conversions to the consumers that export the data.

We met some of these differences in [“Architectural Patterns for Metrics”](#), and Micrometer’s interface is designed to be general enough to accommodate them. For example, not all metrics systems support dimensional annotation of the measurement, and instead, use hierarchical naming.

There is also the issue of *aggregation discipline* that determines how the metrics are created from individual samples. There are two general approaches, and different metrics systems make different decisions about which one to implement:

### *Client-side*

Discrete samples are converted to a fixed rate (aggregated) before they are sent to the server.

### *Server-side*

All samples are sent over the network, and aggregation occurs at the server.

Metrics systems also differ in how the data is actually transmitted (or published) to the server. There are two main options for this as well:

#### *Client push*

The application exporter connects to the server and sends updates to it.

#### *Server poll*

The metrics backend connects to a standard port (usually HTTP) and *scrapes* data from the application.

These aspects are properties of the exporters and are abstracted away by Micrometer. As a result, the developer who wants to work from a Java programming perspective can focus on the Micrometer API and not worry about the details of the metrics backend when implementing metrics in their code.

Within this API, the `Meter` is the key interface for collecting metrics. The different types of meter (the *instrument types*) are represented by instances of classes that implement various subinterfaces of `Meter`. Meters are named using all-lowercase with dot separators, and this is translated, if necessary, into the native naming scheme when exporting metrics.

Each meter lives in a specific *registry*, such as these examples:

#### *SimpleMeterRegistry*

In-memory only, used for development and unit testing

#### *LoggingMeterRegistry*

Also development and testing, but logs meters periodically

#### *CompositeMeterRegistry*

Holds multiple registries (multipub)

#### *Metrics.globalRegistry*

Static global registry

Micrometer autowires a test registry ( `SimpleMeterRegistry` ) when used in Spring apps. Non-Spring applications can simply instantiate an instance of it (and the same is true of `LoggingMeterRegistry` ):

```
// for testing non-Spring applications
MeterRegistry myRegistry = new SimpleMeterRegistry();
```

```
// produces some output
MeterRegistry withOutput = new LoggingMeterRegistry();
```

In our Fighting Animals example (from the `micrometer_only` branch), we want to use a `LoggingMeterRegistry` initially. We wire this up by providing a Spring bean in the application class, like this:

```
@SpringBootApplication
public class AnimalApplication {

    @Bean
    public MeterRegistry basicRegistry() {
        return new LoggingMeterRegistry();
    }

    public static void main(String[] args) {
        SpringApplication.run(AnimalApplication.class, args);
    }
}
```

This will create a new `LoggingMeterRegistry` and make it available for autowiring in the controller. This will log the metrics to the console, which is a reasonable way to get started with Micrometer. Later, you'll see how to build something more sophisticated and similar to what we'd actually use in production.

Note that this plumbing has to be done for each of the `*Application` classes, as the different microservices run in different containers.

Micrometer supports a wide range of instrument types that cover most of the common use cases:

#### *Counter*

Count of all events

#### *Gauge*

Single metric value

#### *Timer*

Count and total time of all timed events

#### *DistributionSummary*

Tracks the distribution of nontimed events (histograms)

Less common instruments include `LongTaskTimer`, `TimeGauge`, `FunctionCounter`, and `FunctionTimer`.

In Micrometer, dimensions are represented as `Tag` objects. Tags are also named using a dotted lowercase convention and must have non-null values.

## Counters

Let's look at a simple Micrometer code example that uses a counter:

```
@RestController
public class AnimalController {
    // ...

    private final Counter battlesTotal;

    private final MeterRegistry registry;

    // ...

    public AnimalController(MeterRegistry registry) {
        this.registry = registry;
        this.battlesTotal = this.registry.counter("battles.total");
    }

    @GetMapping("/battle")
    public String makeBattle() throws IOException, InterruptedException {
        battlesTotal.increment();

        // ...
    }
}
```

A Micrometer `Counter` represents a *monotonic value*—one that can only increase over time. This makes it a good fit for a metric that represents the number of battles that have been fought.

They can be created using the `counter()` method on the registry or by using a fluent builder and `register()`:

```
this.battlesTotal = Counter
    .builder("battles.total")
    .description("Total number of battles fought")
    .register(this.registry);
```

The final step in the creation of a counter is to register it with the registry. There are also some optional methods—such as setting the units and any tags.

# Gauges

Let's look at another example, this time using a `Gauge`. The gauge is a little more complex than the counter, as it needs to be able to change state both up and down rather than just increasing monotonically.

In this case, we want to track the percentage of feline animals that have been seen over time. For this, we need a class that acts as a carrier of a mutable double value. There isn't really anything suitable in the JDK, so we create our own class, `FelinePercent`, which extends `java.lang.Number`.

---

## WARNING

The Java concurrency libraries do not provide an `AtomicDouble` class—which would otherwise have been an obvious choice (for the mutability rather than the concurrency aspect).

---

The resulting code for the controller looks a bit like this (details abridged to make the observability code clearer):

```
@RestController
public class MammalController {
    // ...
    private final FelinePercent felinePercent;
    private int felineCount = 0;
    private int mustelidCount = 0;

    private final MeterRegistry registry;

    public MammalController(MeterRegistry registry) {
        this.registry = registry;
        felinePercent = this.registry.gauge("battles.felinePercent",
            new FelinePercent(0.5));
    }

    @GetMapping("/getAnimal")
    public String getAnimal() throws IOException, InterruptedException {
        // ...

        var id = (int) (SERVICES.size() * Math.random());
        if (id == 0) {
            mustelidCount += 1;
        } else {
            felineCount += 1;
        }
        felinePercent.setValue((double) felineCount / ((double) (felineCount
            + mustelidCount)));
    }
}
```

```

        // ...
    }
}

```

In this example, the key call is to `registry.gauge()`, which creates a new `Gauge` instance and registers it with the registry. In the `Micrometer MeterRegistry` class, the simplest form of the `gauge()` method is this:

```

@Nullable
public <T extends Number> T gauge(String name, T number) {
    return this.gauge((String)name, (Iterable)Collections.emptyList(),
        (Number)number);
}

```

Note that the generics require that the gauge class extends `Number`, so we use this simple class definition:

```

public final class FelinePercent extends Number {
    private volatile double value;

    public FelinePercent(double v) {
        if (v < 0.0 || v > 1.0) {
            throw new IllegalArgumentException("Require 0 < felinePercent < 1");
        }
        value = v;
    }

    public void setValue(double v) {
        if (v < 0.0 || v > 1.0) {
            throw new IllegalArgumentException("Require 0 < felinePercent < 1");
        }
        value = v;
    }

    @Override
    public int intValue() {
        return (int) value;
    }

    @Override
    public long longValue() {
        return (long) value;
    }

    @Override
    public float floatValue() {
        return (float) value;
    }

    @Override

```

```

    public double doubleValue() {
        return value;
    }
}

```

When an instance of `FelinePercent` is passed to the `gauge()` method, it effectively creates a watcher for the state—the percentage of feline animals that have been seen.

The programmer needs to update only the gauge value, and the rest of the metrics system is abstracted away. The watcher updates the gauge value whenever required—but note that this is on demand, so not every state change is necessarily observed, only the current value at the time the update is needed.

## Meter Filters

Micrometer also implements *meter filters*—these provide greater control over:

- How and when meters are registered
- What kinds of statistics they emit

As a first example, filters can be used to adapt metrics to a new (or old) set of conventions without having to make widespread code changes.

Meter filters provide three basic functions:

- Deny/accept meters being registered
- Transform meters (change metric name, tags, units, etc.)
- Configure distribution statistics

Note that the last function can only be configured for appropriate meter types—meaning timers and distribution summaries. We will explain this final function in more detail later in this section when we meet the instrument types that it applies to.

Filters are represented as implementations of the `MeterFilter` interface. They can be added programmatically, often by using a factory method, like this:

```

// This next line prevents the internal metrics from being published
this.registry.config()
    .meterFilter(MeterFilter.denyNameStartsWith("internal"));

```

We don't actually have any internal metrics in our example, but this is a good demonstration of how to use a filter, and preventing internal metrics from being published is a common use case.

---

**NOTE**

`MeterFilter` is an interface, but not a functional interface, as it has three non-static methods, all of which are default methods (it does not have any mandatory methods at all).

---

We can also explicitly construct filter objects. For example, the filter from the previous example is equivalent to this code:

```
new MeterFilter() {
    @Override
    public MeterFilterReply accept(Meter.Id id) {
        if (id.getName().startsWith("internal")) {
            return MeterFilterReply.DENY;
        }
        return MeterFilterReply.NEUTRAL;
    }
};
```

The enum `MeterFilterReply` has three possible values: `DENY`, `NEUTRAL`, and `ACCEPT`. The `DENY` value prevents the meter from being registered, while `ACCEPT` immediately registers it, without looking at any further filters. `NEUTRAL` means that the filter has no opinion on the meter, and the next application filter for this metric (if any) should be consulted.

---

**NOTE**

The implementation of `meterFilter()` is strictly additive, so the developer should consider the order in which filters are added to the chain.

---

Filters can also be used for more advanced use cases, such as using them as customizers for a `CompositeMeterRegistry`. This enables such patterns as sending a subset of metrics to an auxiliary backend. This can be very useful in production deployments, but a full discussion of this is outside the scope of this book.

## Timers

Timers are a more complex data type that store at least three values internally:

- The sum of all recorded values
- A count of the values that have been recorded
- The maximum value seen within a time window, as a gauge

Timers can be configured to emit additional statistics, such as histogram data, precomputed percentiles, or even service level objective (SLO) boundaries.

Let's look at an example and focus on the timer code on the `micrometer_only` branch. Here's the code for the timer usage in the `AnimalController`:

```
@RestController
public class AnimalController {
    // ...

    private final Timer responseTimer;

    private final MeterRegistry registry;

    // ...

    public AnimalController(MeterRegistry registry) {
        this.registry = registry;

        // ...

        this.responseTimer = Timer
            .builder("response.time")
            .description("Response time")
            .register(registry);
    }

    @GetMapping("/battle")
    public String makeBattle() throws Exception {
        Callable<String> callable = () -> {
            // Send the two requests and return the response body as the response
            var good = fetchRandomAnimal();
            var evil = fetchRandomAnimal();
            return String.format("{}"
{ "good": "%s", "evil": "%s" }", good, evil);
            };

            // ...

            return responseTimer.recordCallable(callable);
        }
    }
}
```

In this code, we set up a block of code as a `Callable` and then pass it to the `recordCallable()` method of the timer. This will execute the code block and record the time taken to complete it.

Timers can also handle code represented as `Runnable` and `Supplier`. The appropriate `Timer` method is called `record()` in these cases—this is due to the signature collision between `Callable` and `Supplier`.

To conclude our discussion of Micrometer timers, we should point out that in general, timers are not the preferred approach for measuring the performance of methods in a distributed system. Distributed tracing, such as that implemented by OpenTelemetry, is usually a much better approach for this. We will discuss this approach later in this chapter.

So next, let's discuss the final instrument type we want to look at—the `DistributionSummary`.

## Distribution Summaries

We've just met timers, and they provide statistics on the distribution of the timings that they've seen. In fact, they are a special case of the more general case of distribution summaries.

A *distribution summary* is, as the name suggests, an instrument used to summarize an entire set (or distribution) of values. They require more memory than a simple counter, as they need to store more data, but they still represent a lossy representation of an overall distribution.

---

### TIP

Distribution summaries should be used for things that are not timed. If the measurement quantity is a duration, then a timer should be used instead.

---

Let's look at an example and introduce a `DistributionSummary` into our `AnimalController`:

```
@RestController
public class AnimalController {
    // ...

    private static final Random random = new Random();

    private final DistributionSummary winSummary;

    private final MeterRegistry registry;

    public AnimalController(MeterRegistry registry) {
```

```

        this.registry = registry;

        // Summarizes the size of the attacker's strength when it wins
        this.winSummary = registry.summary("attacker.win.size");

        // ...
    }
}

```

To drive this demonstration of a summary, we'll also introduce some code for a `resolveFight()` method in the `AnimalController`. The aim is to simulate a fight between two animals and return the winner. We normalize the defender's strength to 0.5 and then use a random number for the attacker's strength to determine the winner:

```

@GetMapping("/fight/{a}/{d}")
public String resolveFight(
    @PathVariable("a") String attacker, @PathVariable("d") String defender,
    final String winner;
    // Defender's strength is taken to be 0.5
    var attackerStrength = random.nextDouble();
    if (attackerStrength > 0.5) {
        winner = attacker;
        // Add to the distribution summary
        winSummary.record(attackerStrength);
    } else {
        winner = defender;
    }
    return String.format(
        "{ \"winner\": \"%s\"}", winner);
}

```

If the attacker wins, their “strength” is recorded in the distribution summary. This should produce a uniform distribution of values between 0.5 and 1.0, which the `DistributionSummary` will then summarize.

We'll use the `LoggingMeterRegistry` for this example, and the resulting output looks like this:

```

animal-service_1 | 2024-01-14T08:27:31.748Z INFO 1 --- [tricks-published]
i.m.c.i.logging.LoggingMeterRegistry : attacker.win.size{} ↵
throughput=0.183333/s mean=0.699785 max=0.98829

```

The default setup for distribution summaries is good enough for many purposes, but it is possible to use metric filters to set up more advanced configurations. The key to this is the third of the nonstatic methods in the `MeterFilter` interface, `configure()`, which is defined like this:

```

@Nullable
default DistributionStatisticConfig configure(Meter.Id id,
        DistributionStatisticConfig config) {
    return config;
}

```

The default implementation represents an identity transformation of the configuration, but in general, a custom implementation will merge the supplied configuration with the input configuration.

By defining a custom filter that overrides this method, it is possible to configure optional distribution statistics (in addition to the basics of count, total, and max). These additional statistics can include precomputed percentiles, SLOs, and histograms.

For example, to configure precomputed “long-tail” percentiles (which we met in [Chapter 2](#)) for all JVM metrics, we could use a filter like this:

```

new MeterFilter() {
    @Override
    public DistributionStatisticConfig configure(Meter.Id id,
        DistributionStatisticConfig config) {
        if (id.getName().startsWith("jvm")) {
            return DistributionStatisticConfig.builder()
                .publishPercentiles(0.9, 0.99, 0.999, 0.9999)
                .build()
                .merge(config);
        }
        return config;
    }
};

```

This will add the 90th, 99th, 99.9th, and 99.99th percentiles to all JVM metrics—which can be very useful to observe the non-normal distribution of many of the JVM metrics.

It is important at this point to note that count, sum, and some other data associated with distribution summaries can be reaggregated across dimensions (or even across instances). However, precomputed percentile values cannot be reaggregated—to attempt to do so is a serious and all-too-common error.

The reason is that the way percentiles are created makes them unique to each dataset. Accurate percentiles across the entire dataset can only be computed by combining the original datasets and *then* calculating the percentiles. Once percentiles have been calculated, data has already been

lost, and reaggregating the percentiles will not produce the correct result except in pathological special cases.

To conclude this discussion of Micrometer, let's take a quick look at the JVM metrics support that the library provides out of the box.

## Runtime Metrics

In addition to programmer-defined metrics, Micrometer also provides the capability to collect and export a set of metrics associated with the JVM and other parts of the application runtime. There are a number of different sets of this type of metric that can be collected.

The key interface for this is `MeterBinder`, which is defined like this:

```
public interface MeterBinder {  
    void bindTo(@NonNull MeterRegistry var1);  
}
```

Two of the most important implementations of this are the JVM memory metrics and the processor metrics, as shown in this example:

```
@RestController  
public class AnimalController {  
    // ...  
  
    private final MeterRegistry registry;  
  
    // ...  
  
    public AnimalController(MeterRegistry registry) {  
        this.registry = registry;  
  
        new ProcessorMetrics().bindTo(this.registry);  
        new JvmMemoryMetrics().bindTo(this.registry);  
    }  
  
    // ...  
}
```

The key to this is the `bindTo()` method, which makes the registry aware of the JVM-level metrics. It is not strictly necessary for Spring Boot applications, as the framework auto-installs them, but for other applications, they must be explicitly enabled. Other sets of metrics can be collected as well, but these are some of the most common ones.

To give some specific examples, the `JvmMemoryMetrics` class provides metrics such as `jvm.memory.used` and `jvm.memory.max`, while `ProcessorMetrics` provides metrics like `system.cpu.usage` and `system.load.average.1m`. One other important case is that of metrics from an `ExecutorService`, which can be enabled like this: `new ExecutorServiceMetrics(executor, executorServiceName, tags).bindTo(registry)` and enables easy monitoring of thread pools.

Having met the basic instruments and functionality provided by Micrometer, let's move on to meet Prometheus properly and see how we can integrate it with Micrometer.

## Introducing Prometheus for Java Developers

We introduced Prometheus very briefly in [Chapter 8](#) and made several references to it in the last chapter, but we didn't do a thorough treatment of the technology. In this section, we will discuss it in more detail, in particular, how it can be used by Java developers and projects.

### Prometheus Architecture Overview

To recap, Prometheus is a CNCF project (originally created at SoundCloud) that provides a metrics backend, collection mechanism, and various integrations. It is designed to handle purely numeric, regular time series data and is not intended to be used for things like logs or traces.

Of the metrics architectural options discussed before, Prometheus uses server poll, which it refers to as scraping. This means that every service you wish to monitor using Prometheus must provide an HTTP endpoint from which the Prometheus scraper will collect metrics. In turn, this means that Prometheus also relies on services being known or discoverable, which potentially presents issues for short-lived jobs.

To handle these challenges, and also to integrate better with technologies like OpenTelemetry that use push-based architectures, Prometheus also ships a remote write capability. This is also more in accordance with the security model of systems like Kubernetes.

The overall architecture of a (relatively complex) Prometheus deployment can be seen in [Figure 11-1](#).

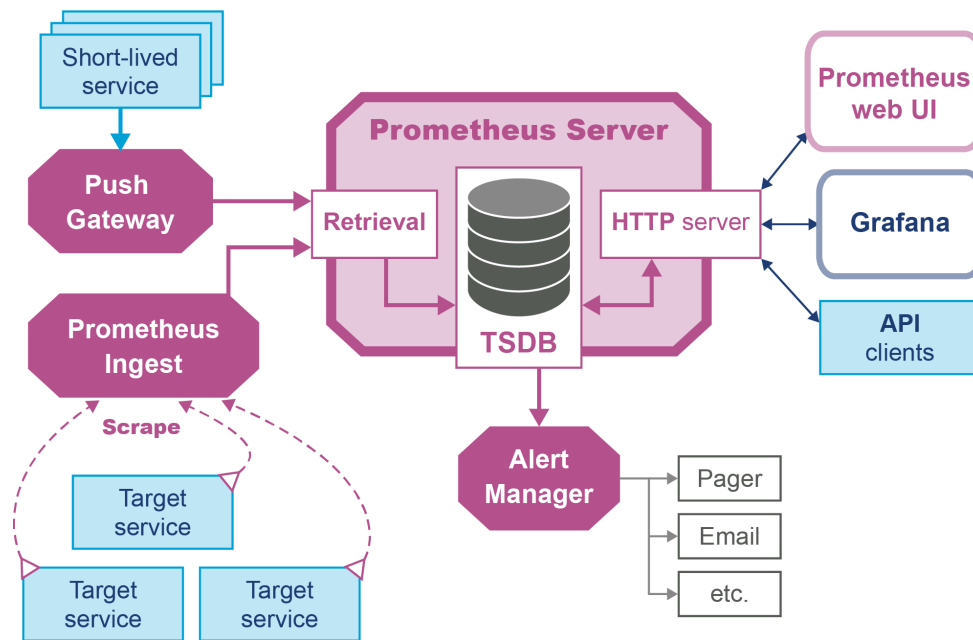


Figure 11-1. Prometheus architecture (source: [Prometheus documentation](#))

As the diagram makes clear, there are a large number of components, and the exact combination will depend upon the architectural choices made. Thus it is important to ask for clarification when people state that they are “using Prometheus.”

Prometheus provides a query language known as PromQL, which is used to write queries against the collected data. Despite the name, PromQL is not SQL, and is instead a domain-specific language (DSL) designed to be used for querying time series data rather than traditional relational data. The queried data can then be visualized in a number of different ways.

Prometheus ships with a rudimentary UI, but this is frequently insufficient for production use. Instead, it is more common for other tools to layer on top of Prometheus—the open source Grafana graphing tools are a popular choice.

In general, developers and DevOps folk typically interact with Prometheus in two ways—either via the UI or by calling the metrics collection code from within their app code. However, a general awareness of the overall architecture of Prometheus (including its data storage capabilities) is useful for understanding how it fits into the overall observability picture—even if you’re not directly responsible for maintaining the deployment.

## Using Prometheus with Micrometer

In our initial examples, we used the `LoggingMeterRegistry` to output metrics to the console. This is, of course, not a realistic production configuration. In this section, we will see how we can use Prometheus as a metrics backend for Micrometer.

Recall that we want to use a facade pattern to abstract away the details of the metrics backend. The idea is that there should be no Prometheus-specific code in our application. This makes testing easier, as it can be run with a dummy or mocked metrics dependency.

It is sometimes said that use of facades means that it's possible to swap out components, such as replacing Prometheus for another metrics backend without changing our application code. This ability—to change the observability plumbing without requiring application changes—can be really important, and it is one of the major drivers for standardization.

Having said this, there are often details that make this not as straightforward in practice—vendor lock-in can be more subtle than we expect. It is also true that in many cases a major change of components is accompanied by an opportunity to revisit the overall architecture and make other changes as well.

Nevertheless, the facade nature of Micrometer's architecture makes the use of Prometheus as a metrics backend relatively straightforward. To implement Prometheus in our applications, we can use the SPI nature of the Micrometer library. All we need to do is to add another dependency to our project:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-prometheus</artifactId>
  <scope>runtime</scope>
</dependency>
```

This provides an additional exporter that can be used to send metrics to Prometheus.

To see this in `fighting-animals`, we need to make a few changes. To make this clearer, this is done on a separate branch (`micrometer_with_prom`).

We need to add this line to `application.properties`:

```
management.endpoints.web.exposure.include=health,info,prometheus
```

We are using Micrometer to directly expose the metrics from the application as a scrapeable endpoint that Prometheus can connect to.

You can also remove the logging registry to cut down on the amount of detail in the logs:

```
// Remove this bean to reduce log noise
@Bean
public MeterRegistry basicRegistry() {
    return new LoggingMeterRegistry();
}
```

The resulting architecture here is still fairly simple, as seen in [Figure 11-2](#).

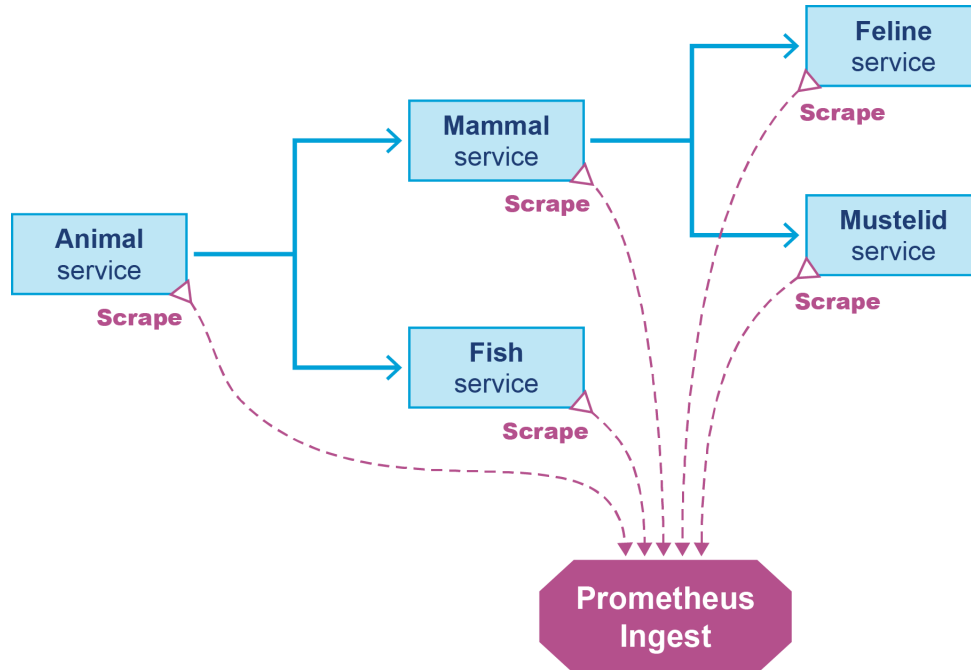


Figure 11-2. Fighting Animals with Prometheus

To configure Prometheus for this architectural configuration, we need to add a new service to our `docker-compose.yml` file:

```
# Prometheus
prom:
  container_name: prometheus
  image: prom/prometheus
  command:
    - "--config.file=/config/prometheus.yml"
  ports:
    - "9090:9090"
  user: root
  volumes:
    - './config:/config'
    - './target/data/prometheus:/prometheus'
```

This refers to a new configuration file, `prometheus.yml`, the bare bones of which look like this:

```
global:
  # Set the scrape interval to every 15 seconds. Default is every 1 minute
  scrape_interval: 15s
```

```

# Evaluate rules every 15 seconds. The default is every 1 minute.
evaluation_interval: 15s
# scrape_timeout is set to the global default (10s).

# ...

scrape_configs:

- job_name: prometheus
  metrics_path: /metrics
  scheme: http
  static_configs:
  - targets:
    - localhost:9090

- job_name: animal
  metrics_path: /actuator/prometheus
  scrape_interval: 5s
  static_configs:
  - targets:
    - animal-service:8080

# Other services similarly configured

```

This is a simple configuration, but it has been tested to work across a real network rather than with everything running on `localhost`. This is quite deliberate, as many examples available on the internet do not work in a real network environment—only on `localhost`.

With our config, each service will expose its metrics on the same URI (`/actuator/prometheus`) on a different port, and Prometheus will scrape them all. Typical Prometheus output looks like this (from the animal service, deployed at a URL like `http://<Target IP>:8080/actuator/prometheus`):

```

# HELP system_load_average_1m The sum of the number of runnable entities q
to available processors and the number of runnable entities running on the
available processors averaged over a period of time
# TYPE system_load_average_1m gauge
system_load_average_1m 0.2
# HELP process_files_open_files The open file descriptor count
# TYPE process_files_open_files gauge
process_files_open_files 34.0
# HELP jvm_classes_loaded_classes The number of classes that are currently
loaded in the Java virtual machine
# TYPE jvm_classes_loaded_classes gauge
jvm_classes_loaded_classes 8934.0
# HELP tomcat_sessions_active_current_sessions
# TYPE tomcat_sessions_active_current_sessions gauge
tomcat_sessions_active_current_sessions 0.0
# HELP jvm_memory_committed_bytes The amount of memory in bytes that is ↵

```

```

committed for the Java virtual machine to use
# TYPE jvm_memory_committed_bytes gauge
jvm_memory_committed_bytes{area="nonheap",id="CodeHeap 'profiled nmethods'
  9109504.0
jvm_memory_committed_bytes{area="heap",id="G1 Survivor Space",} 4194304.0
# ... Other JVM metrics omitted for brevity ...
jvm_memory_committed_bytes{area="nonheap",id="CodeHeap 'non-profiled nmethods'
  3145728.0

```

Note that many of these metrics are JVM metrics rather than application metrics. If we look further down the output, we can see some of our custom metrics:

```

# HELP battles_total
# TYPE battles_total counter
battles_total 5.0

```

Prometheus monitors itself using the same mechanism, so we can see the Prometheus metrics as well: `http://<Target IP>:9090/metrics`:

```

# HELP go_gc_duration_seconds A summary of the pause duration of garbage
collection cycles.
# TYPE go_gc_duration_seconds summary
go_gc_duration_seconds{quantile="0"} 3.5e-05
go_gc_duration_seconds{quantile="0.25"} 7.8507e-05
go_gc_duration_seconds{quantile="0.5"} 9.9929e-05
go_gc_duration_seconds{quantile="0.75"} 0.000132907
go_gc_duration_seconds{quantile="1"} 0.000325268
go_gc_duration_seconds_sum 0.018079852
go_gc_duration_seconds_count 164
# HELP go_goroutines Number of goroutines that currently exist.
# TYPE go_goroutines gauge
go_goroutines 47
# HELP go_threads Number of OS threads created.
# TYPE go_threads gauge
go_threads 18
# HELP go_info Information about the Go environment.
# TYPE go_info gauge
go_info{version="go1.17.5"} 1
# ....
# HELP net_conntrack_dialer_conn_attempted_total Total number of connections
attempted by the given dialer a given name.
# TYPE net_conntrack_dialer_conn_attempted_total counter
net_conntrack_dialer_conn_attempted_total{dialer_name="alertmanager"} 0
net_conntrack_dialer_conn_attempted_total{dialer_name="animal"} 42
net_conntrack_dialer_conn_attempted_total{dialer_name="default"} 0
net_conntrack_dialer_conn_attempted_total{dialer_name="feline"} 41
net_conntrack_dialer_conn_attempted_total{dialer_name="fish"} 41
net_conntrack_dialer_conn_attempted_total{dialer_name="mammal"} 42

```

```
net_conntrack_dialer_conn_attempted_total{dialer_name="mustelid"} 42
net_conntrack_dialer_conn_attempted_total{dialer_name="prometheus"} 2
```

The first group of metrics are the Go runtime metrics, including GC metrics and goroutine counts. As you can see, Prometheus is written in Go, and the Go language supports *goroutines*—lightweight threads that are managed by the Go runtime. These are very similar to the Java concept of virtual threads, which we will discuss in [Chapter 14](#).

This provides an example of an important architectural principle that Prometheus exemplifies—the systems that support and convey observability signals should themselves be observable and well-architected from an observability perspective.

The basic Prometheus UI can be found at: `http://<Target IP>:9090/graph` and is shown in [Figure 11-3](#).

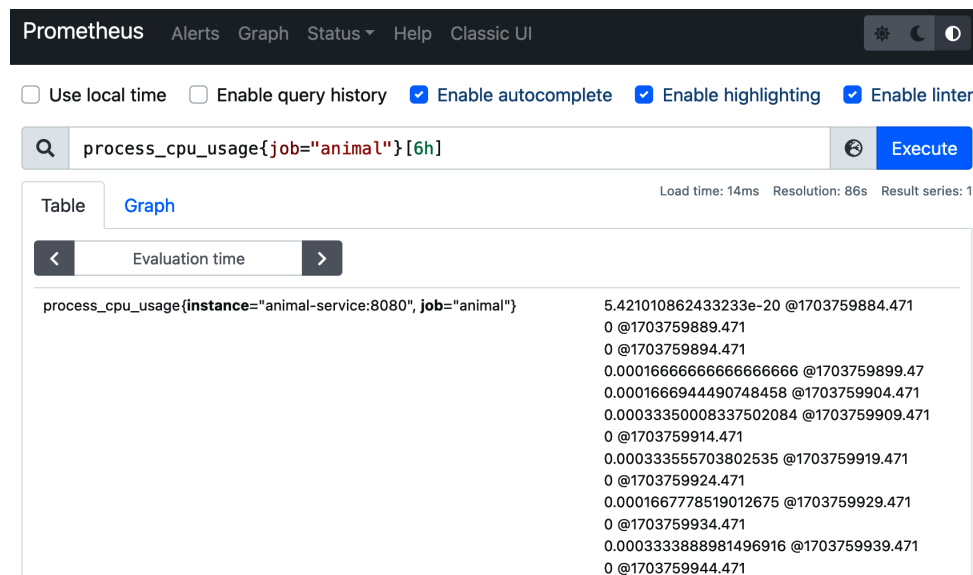


Figure 11-3. Prometheus query UI

Our query string here is `process_cpu_usage{job="animal"}[6h]`, which is a PromQL query that asks for all the data points representing the CPU usage of the `animal` job over the last six hours. This is known as a *range vector*, as it returns a vector of time series over a range of time.

Note that this is displayed here in the Table view, in the form: `<value> @<timestamp>`. If we switch to the graphing view, then we need to change the query, as graphing requires an expression that is an *instant vector*.

The time series for the query `process_cpu_usage{job=~"m.*"}` is depicted graphically in [Figure 11-4](#). This query returns the CPU metric for jobs that start with the letter `m`—in our case `mammal` and `mustelid`. This uses the Prometheus regular expression support, `~`, which is very useful for selecting metrics.

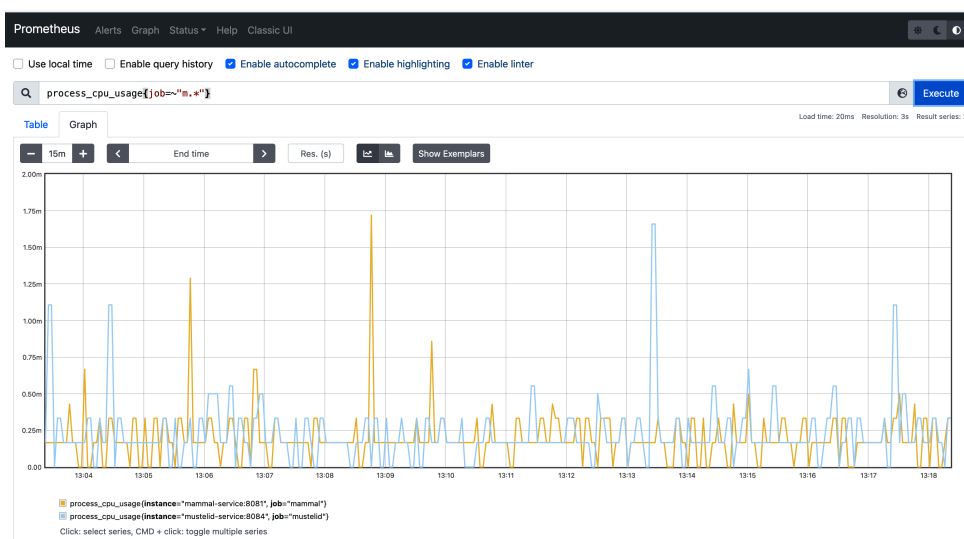


Figure 11-4. Prometheus graphs

A full discussion of PromQL is outside the scope of this book—the Prometheus documentation should be consulted for any serious work, as well as that for auxiliary components such as Grafana.

Instead, let’s move on to discuss the third, and arguably most important, of our observability technologies—OpenTelemetry.

## Introducing OpenTelemetry

In this section, we’ll properly introduce OpenTelemetry (aka OTel), a new open standard from the CNCF for observability data, which we mentioned in [Chapter 8](#). The project was formed by a merger of the OpenTracing (tracing) and OpenCensus (metrics) projects and is now an open standard that is rapidly gaining traction, showing rapid growth in adoption.

---

### NOTE

OpenTracing and OpenCensus are now deprecated, and all new development should be done using OpenTelemetry.

---

This growth is even more impressive when you consider that the project is only a few years old and only reached its 1.0 release during 2023. The project is now stable and ready for production use and has been enthusiastically adopted by many companies and teams already—and that trend looks set to continue.

OTel can be used in a wide range of circumstances, but it is perhaps most at home in cloud-deployed, microservice-based applications, and it is generally suitable for heterogeneous (aka polyglot), distributed systems.<sup>1</sup>

# What Is OpenTelemetry?

One of the key strengths of OTel is that it does not try to provide a solution for every problem but focuses on its core domain. OTel is *not* a data ingest or observability backend, and thus, is only one component of a complete observability system.

As we mentioned briefly in [Chapter 10](#), OTel is focused on instrumentation of applications and the transport of data to a separate, external observability system.

The major project areas of OpenTelemetry are shown in [Figure 11-5](#).

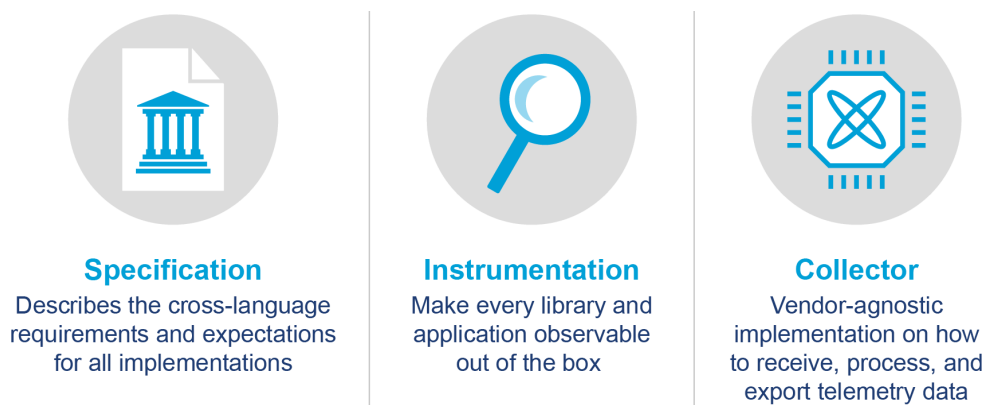


Figure 11-5. Concerns of OpenTelemetry project

The specification defines formats and conventions for metrics, logs, and traces in a language-independent way and a protocol for transporting the data to the observability backend.

The various language implementations provide a set of APIs and SDKs that can be used to instrument applications written in virtually any mainstream language, and integrations to a wide range of frameworks. The Java implementation also provides a Java agent to instrument applications without needing to change code.

Finally, the OpenTelemetry Collector is a useful tool that can collect data from applications and send it to a variety of different backends, after having potentially performed data processing and enrichment. You can think of it as a router or “switching station” that can also act as a protocol translator for observability data.

These design choices allow OTel to support various observability data architecture options—it is intended to be widely applicable and not prescriptive.

At a software level, the components provided for instrumentation can be seen in [Figure 11-6](#).

## OpenTelemetry Components

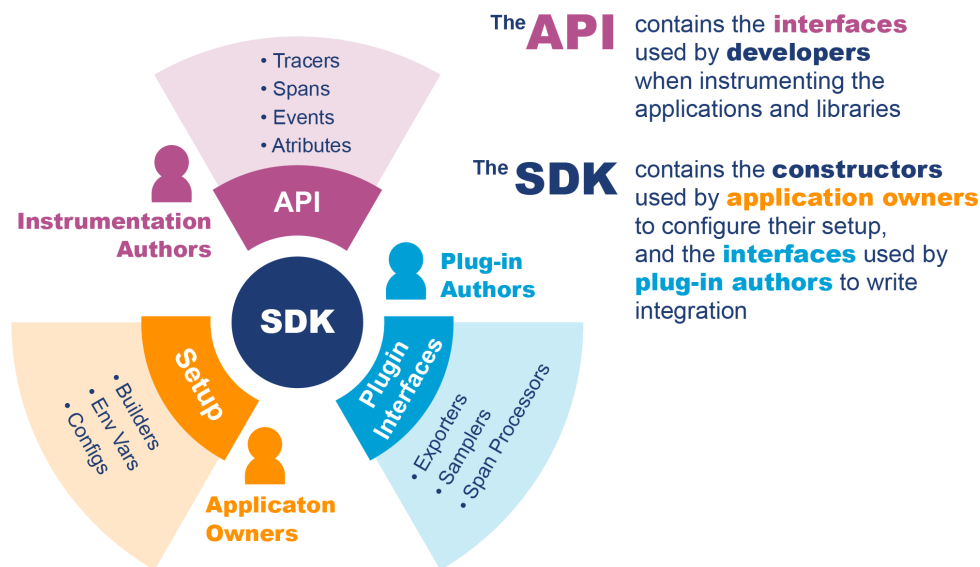


Figure 11-6. OpenTelemetry APIs and SDKs

The SDK is the primary aspect of OpenTelemetry that end users will interact with. It contains two main components:

- Constructors used by application owners to configure their deployment
- Interfaces used by plug-in authors to write integrations

This is the piece that teams deploying OpenTelemetry out of the box will use.

The API contains interfaces used by developers to write bespoke instrumentation for their apps and libraries.

---

### NOTE

If an engineer wants to contribute code to the OpenTelemetry project, then the appropriate starting point for their contributions is usually providing some new instrumentation for an open source library or framework that does not currently have specific OTel support.

---

OTel places great emphasis on stability and backward compatibility. The code follows [\*semantic versioning\*](#) conventions and provides, at minimum, long-term support for the stable versions of the API and SDK:<sup>2</sup>

#### *API*

Three-year support guarantee

#### *Plug-in interfaces*

One-year support guarantee

#### *Constructors*

For Java, there are four main projects under the GitHub `open-telemetry` org:

*opentelemetry-java*

Core components including the API and SDK

*opentelemetry-java-instrumentation*

Library instrumentation and auto instrumentation agent

*opentelemetry-java-contrib*

Helpful and standalone libraries

*opentelemetry-java-examples*

Manual instrumentation examples<sup>3</sup>

The Java implementation initially focused on traces and metrics, with logs reaching 1.0 in late 2023.

## Why Choose OTel?

The application performance monitoring (APM) market was originally dominated by proprietary vendors. As with many other market segments, demand for open source alternatives emerged—to reduce vendor lock-in, cut costs, and provide greater flexibility.

We have already met some of these open source projects, such as Jaeger for traces, Prometheus in the metrics space, and the ELK stack for logging.

A second market trend is more subtle, but no less important—the increasing complexity of software systems. As software continues to become more complex, and as more languages and frameworks become popular, more and more resources are required to provide a credible instrumentation.

For proprietary observability products, this trend creates duplication and inefficiency, as each separate company must maintain its own suite of instrumentation libraries. This duplicated effort ultimately causes it to make more sense for observability vendors to collaborate on a single set of open source libraries rather than each maintaining its own. The value that an observability vendor provides is then in the user experience, backend capabilities, and cost rather than the instrumentation per se.

This represents a switch from proprietary to open source—at least, within the domain of the code that runs in the context of a customer’s applica-

tion. In recent years, we have indeed seen a number of proprietary vendors switch to an open source model.

This can be seen as an attempt by APM vendors to redefine themselves as observability providers. This has also been accompanied by an increasing number of observability startups that have always been more closely aligned with the open source model.

The protocol and instrumentation stack that vendors are converging on is, of course, OpenTelemetry.

The standard has been criticized for being slow moving, but the benefit of having stable, common naming conventions and associated semantic conventions is huge. In that light, it is perhaps no surprise that more and more companies and teams are adopting OpenTelemetry.

## OTLP

A key part of the OpenTelemetry project is the OpenTelemetry Protocol (OTLP). OTLP does not try to define the entire protocol space, and instead focuses on its key concerns:

- Encoding
- Transport
- Delivery

Performance is a key consideration for OTLP, and it is typically implemented using HTTP/2 or [gRPC](#)—which is essentially a *remote procedure call* (RPC) framework that uses the binary *Protocol Buffers* (protobuf) format over HTTP/2.<sup>4</sup> The core Java implementation can use either encoding and defaults to HTTPS/protobuf.

Before we start looking at the Java libraries that provide the SDK and APIs for OTel, we need to introduce a very important component—the OpenTelemetry Collector.

## The Collector

The OTel Collector is a network service that works with streams of observability data and can receive, process, and export any or all of the three main types of observability data.

The simple architecture of the Collector is designed to be extensible and is vendor neutral. It is written in Go and is maintained by an open source team drawn from a variety of different companies. Despite the name, it also works with a wide variety of data formats, not just OTLP.

The Collector is configured using YAML and deploys to `http://localhost:4317` by default. The main configuration sections for the Collector are:

#### *receivers*

Data sources that the Collector will listen to data from

#### *processors*

Data transformations that the Collector will apply

#### *connectors*

Optional components that transform one type of telemetry to another

#### *exporters*

Where to send the data after transformation

#### *extensions*

Any optional components (e.g., health-checking)

There is also a `service` section that is used to configure the pipelines of the Collector—it has separate sections for each observability signal that we want to handle. For example, this `service` section shows a simple trace pipeline:

```
service:
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [otlphttp]
```

Data will be accepted from the `otlp` receiver, processed by the `batch` processor, and then sent to the `otlphttp` exporter—and the data will be processed into batches before being sent. Note that, just like Prometheus, the Collector generates telemetry about itself in the (somewhat confusingly named) `telemetry` section beneath `service`.

The core architectural principles of the Collector are:

#### *Usability*

Reasonable default config and should work out of the box

#### *Performance*

Performant under varying loads and configurations

### *Observability*

Good example of an observable service

### *Extensibility*

Customizable without touching the core code

### *Unified*

Single codebase supports traces, metrics, and logs

In [Figure 11-7](#) we show a sample architecture that uses OpenTelemetry to handle both metrics and traces. We can see that in this view, the Collector can be thought of as a shim between application processes (including short-lived) and data storage for metrics and traces.

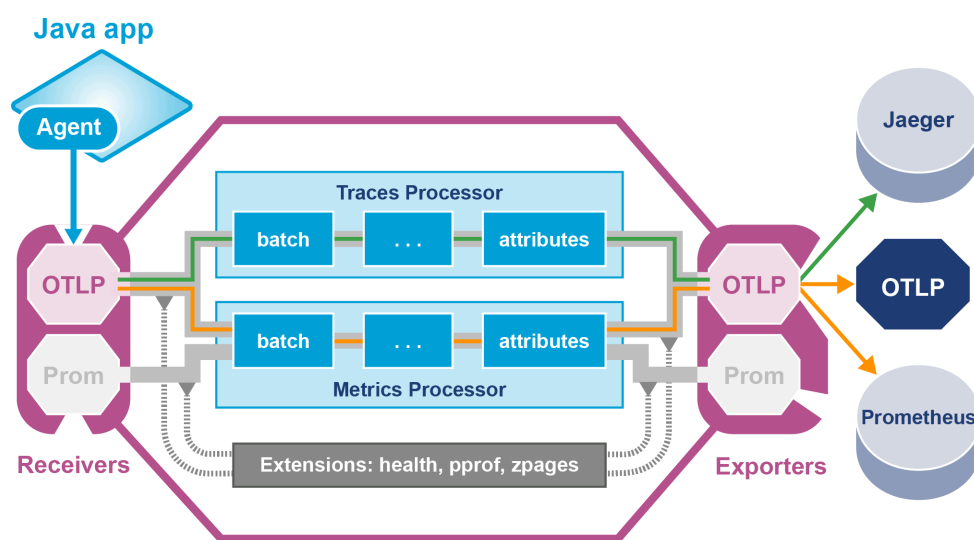


Figure 11-7. OpenTelemetry example architecture

In this example, the Collector instance receives data from multiple processes and exposes a single endpoint. For the case of metrics, this maintains flexibility of architectural choice—Prometheus either knows where the collectors are located, or the collectors can be configured to remote-write.

The Collector is actually a fairly simple, but very flexible, component in terms of configuration and operation, and we will return to it once we've introduced the Java libraries that will be sending data to it.

## OpenTelemetry Tracing in Java

As we discussed earlier—OTel focuses on its core domain, which is instrumentation and data exfiltration. These are low-level concerns, so while OpenTelemetry does provide user-facing APIs, there are a number of approaches to adopting OpenTelemetry, some of which avoid explicit coupling to those APIs.

In addition, OTel does not attempt to provide a single unified API for all the different types of observability data in Java. Instead, where possible, it seeks to integrate with Java components that already exist and are widely used in the ecosystem.

As we'll see later, for logs, good patterns (and high-level APIs) exist, and OTel best practice is to use them. For traces, however, there is no existing candidate of suitable quality, so OpenTelemetry provides two different solutions—manual or automatic tracing.

Before we look at these two approaches, a quick word about OpenTelemetry architecture. It is designed to be used in a production deployment, so a certain amount of configuration is required, and a certain amount of extra complexity is unavoidable.

The following subsection, on manual tracing, is, therefore, longer than it really warrants. This is because it has to introduce infrastructure components (such as the Collector) and changes to the POM to bring in the required OTel dependencies.

## Manual Tracing

As the name implies, manual instrumentation requires the developer to manually insert explicit calls to the tracing library. This means that while it is possible to use it for tracing, in practice, anything but a trivial example quickly becomes too complex to manage.

This is best shown with an example—so let's look at how we'd use manual tracing in Fighting Animals. In this section, we'll be looking at code taken from the `manual_tracing` branch.

First of all, we need to depend upon the OpenTelemetry libraries directly in our project. The necessary changes to the POM are quite extensive—the key part is the addition of the `<dependencyManagement>` section and the use of a BOM, essentially to manage the dependencies as a group:

```
<dependencyManagement>
  <dependencies>
    <dependency>
      <groupId>io.opentelemetry</groupId>
      <artifactId>opentelemetry-bom</artifactId>
      <version>1.40.0</version>
      <type>pom</type>
      <scope>import</scope>
    </dependency>
  </dependencies>
</dependencyManagement>
```

We can then pull in additional OTel dependencies as required—we need the `opentelemetry-api`, `opentelemetry-sdk`, `opentelemetry-sdk-extension-autoconfigure`, and `opentelemetry-exporter-otlp` dependencies for this example.

We've also introduced an auto-configured `OpenTelemetry` bean in `AnimalApplication`, like this:

```
@Bean
public OpenTelemetry openTelemetry() {
    return AutoConfiguredOpenTelemetrySdk.initialize().getOpenTelemetrySdk()
}
```

This `OpenTelemetry` object is now available for use in our code, via constructor injection into the service controllers.

Now, let's look at the code changes required to add tracing to the main HTTP route from the `AnimalController` class:

```
@GetMapping("/battle")
public String makeBattle() throws IOException, InterruptedException {
    // Extract the propagated context from the request. In this case,
    // no context will be extracted from the request - this is the root span
    var extractedContext = extractContext(httpServletRequest, EXTRACTOR);

    try (var scope = extractedContext.makeCurrent()) {
        // Start a span
        var span = serverSpan("/battle", HttpMethod.GET.name(),
            AnimalController.class.getName(), "animal-service:8080");

        // Send the two requests and return the response body as the response
        // and end the root span.
        try {
            var good = fetchRandomAnimal(span);
            var evil = fetchRandomAnimal(span);
            return String.format("""
{ "good": "%s", "evil": "%s" }""", good, evil);
        } finally {
            span.end();
        }
    }
    // ...
}
```

This is only one service, of course. Very similar tracing code must be added to every service if we are to avoid gaps in the tracing coverage of our code. For example, we need to modify `MammalController` like this:

```

@GetMapping("/getAnimal")
public String makeBattle() throws IOException, InterruptedException {
    // Context will be extracted from that propagated from the Animal Serv:
    var extractedContext = extractContext(httpServletRequest,EXTRACTOR);

    try (var scope = extractedContext.makeCurrent()) {
        var span = serverSpan("/getAnimal", HttpMethod.GET.name(),
            MammalController.class.getName(), "mammal-service:8081");

        // Send the sub-request, return the response and end the span
        try {
            return fetchRandomAnimal(span);
        } finally {
            span.end();
        }
    }
}

```

Both of these controllers use the `serverSpan()` method, which is defined in the `Misc` helper class in our application:

```

public static Span serverSpan(Tracer tracer, String path, String method,
    String serviceName) {
    return tracer
        .spanBuilder(path)
        .setSpanKind(SpanKind.SERVER)
        .setAttribute(SemanticAttributes.HTTP_METHOD, method)
        .setAttribute(SemanticAttributes.HTTP_SCHEME, "http")
        .setAttribute(SemanticAttributes.HTTP_HOST, serviceName)
        .setAttribute(SemanticAttributes.HTTP_TARGET, path)
        .startSpan();
}

```

The `Tracer` object is akin to a logger, so it's best to initialize it once, in the constructor code. Having created spans, we need somewhere to send them, so in our `docker-compose.yml`, we have a `Collector` instance configured to receive them and then forward them on to Jaeger:

```

# Jaeger
# Local GRPC port (4317) needs to be remapped to appear as 14317
# to avoid a clash with the OTel collector's GRPC port
jaeger-all-in-one:
    image: jaegertracing/all-in-one:1.52.0
    ports:
        - "16686:16686"
        - "14317:4317" # OTLP gRPC receiver
        - "4318:4318" # OTLP HTTP receiver

# Collector

```

```
otel-collector:
  image: otel/opentelemetry-collector:0.91.0
  command: [ "--config=/etc/otel-collector-config.yaml" ]
  volumes:
    - ./otel-collector-config.yaml:/etc/otel-collector-config.yaml
  ports:
    - "13133:13133" # Health_check extension
    - "4317:4317"   # OTLP gRPC receiver
    - "55681:55681" # OTLP HTTP receiver alternative port
  depends_on:
    - jaeger-all-in-one
```

As per [“A Word About Version Numbers”](#), we use explicit version numbers to ensure that the example will work as is. For actual production usage, you should upgrade to a more recent version of these components before deployment, as older images may harbor security vulnerabilities or other bugs.

We also need to configure the Collector to send the spans to Jaeger, which we do in `otel-collector-config.yaml`:

```
receivers:
  otlp:
    protocols:
      grpc:
      http:

exporters:
  otlphttp:
    endpoint: http://jaeger-all-in-one:4318

processors:
  batch:

extensions:
  health_check:

service:
  extensions: [health_check]
  pipelines:
    traces:
      receivers: [otlp]
      processors: [batch]
      exporters: [otlphttp]
```

These components are essentially infrastructure—they are not part of the application code, but are instead part of the observability infrastructure. This also provides a useful indirection point. The operations folks can

make changes to the configuration of the Collector without needing to change the application.

We also need to tell the microservices where to find the Collector, as it's now running in its own container—in our example, we do this using a Java command-line parameter to set a system property: –  
`Dotel.exporter.otlp.endpoint=http://otel-collector:4317/`,  
which we include in the Dockerfile. This configuration can also be set using environment variables, such as `OTEL_EXPORTER_OTLP_ENDPOINT`. These could also be configured in the Dockerfile.

Just from the two controller classes we met in this section, we can see that the amount of code required to add tracing to our application is quite substantial. This provides a good example of what we mentioned in [“Manual Versus Automatic Instrumentation”](#): the complexity of manual tracing quickly outstrips the ability of the programmer to manage it.

The solution to this sort of tedious, meticulous complexity is the same as it always is—we make the computer do it instead.

## Automatic Tracing

This leads us to automatic instrumentation, which, in the case of OpenTelemetry, usually means deploying with an agent that can work with any Java 8+ application. Dynamic attachment of the agent is also possible and may be preferred in some circumstances. Finally, some frameworks, such as Quarkus, have built-in support and do not require—and should not use—the agent.

---

### NOTE

As we discussed briefly in [Chapter 3](#), a Java agent is a special jar file that contains code to be run before the main method is called. Building such things is an advanced topic, and the interested reader is advised to consult specialist material.

---

The agent adds extra bytecode to provide the timings of methods and other information that is needed to build a trace. In particular, this includes the trace ID and the span IDs, which will be used to link the spans together into a trace. The spans will be stored in memory temporarily, and then the OpenTelemetry exporters will send them to the backend.

---

#### NOTE

In addition to the instrumentation of core code and exporting by the agent, the `opentelemetry-java-instrumentation` project has modules that support over 100 of the most popular libraries and frameworks out of the box.

---

The branch `auto_tracing_only` of Fighting Animals contains an example of automatic tracing in action. It has a prebuilt agent jar available as part of the repo, and the location of this jar file must be passed to the JVM as a command-line argument, which you can see in the Dockerfile.

One major advantage of automatic tracing is that it is much easier to use than manual tracing, and it also keeps the project codebase free of explicit compile-time dependencies on OTel. You can see that by looking at this branch—there are no explicit dependencies on OTel in the POM, and no explicit code to create spans.

By default, the agent uses an OTLP exporter and points at `http://localhost:4317`, where it expects a local OpenTelemetry Collector to be running.

However, on this branch, the Collector setup is exactly the same as for manual tracing, so we won't repeat it here. So, just as we did for manual tracing, we need to add a command-line parameter to tell the application where to find the collector: –

```
Dotel.exporter.otlp.endpoint=http://otel-collector:4317/ .
```

The use of automatic tracing is a good example of the principle of *separation of concerns*—the application code is not concerned with the details (or even existence) of tracing, and the tracing capability is provided by the Java agent and some infrastructure configuration.

Before we move on from the subject of distributed tracing, there's one practical aspect that we still need to discuss—sampling.

## Sampling Traces

In [“Interpretation of Statistics”](#), we met the “hat/elephant” problem, a whimsical name for a very real problem: not all response categories have the same information content. For example, unless there are obvious degradations in response time, successful responses are not very interesting.

The same is true of traces—the vast majority of traces are successful, so are not very interesting. On the other hand, traces that are slow or fail are much more interesting, so we want to be able to see them.

*All happy families are alike; each unhappy family is unhappy in its own way.*

—Leo Tolstoy, *[Anna Karenina](#)*

One solution that the community has adopted is to vary the rate of sampling of traces, depending on the response code, and the number of transactions a service sees. In general, we always want to capture 100% of all errors (whether 4xx or 5xx errors) and then sample a percentage of the successful traces. For relatively high-volume services, this can be as low as 1% of successful traces.

This works because, if we have a high-volume service, then we will still get a statistically significant sample of successful traces, so any performance regressions will still be visible in the sampled data.

## OpenTelemetry Metrics in Java

Let's turn our attention to OTel metrics. There is a manual API provided for handling metrics using the low-level OTel constructs. We'll see how this compares to Micrometer presently, but for the sake of completeness, let's look at a quick example from the `otel_metrics_raw_api` branch of Fighting Animals.

This branch has explicit dependencies on OTel libraries, just as for manual tracing. So, the POM changes are similar, and we also need the `opentelemetry-sdk-metrics` to be available.

Note that we have also removed the `spring-boot-starter-actuator` dependency. This is necessary to avoid conflicts with Spring Boot's bundled Micrometer, and we removed the bean providing a `MeterRegistry` bean. Instead, we're relying on the `OpenTelemetry` bean, just as we did for manual tracing.

In the code, we're using the instruments from the OTel metrics API in the package `io.opentelemetry.api.metrics` to handle our metrics. Let's look at an example in the `AnimalController`, which uses a `LongCounter` and an `ObservableDoubleGauge`.

First, let's declare our fields—a copy of the `OpenTelemetry` bean and the metrics we want to use:

```
public class AnimalController {  
    // ...  
  
    private final OpenTelemetry sdk;
```

```

private final Meter appMeter;
private final Meter memoryMeter;
private final LongCounter battlesTotal;
private final ObservableDoubleGauge cpuTotal;

```

Next, in the constructor, we stash the `OpenTelemetry` bean in a field, and then use it to create the metrics we want to use:

```

public AnimalController(OpenTelemetry sdk) {
    this.sdk = sdk;

    Meter appMeter = sdk.getMeter(INSTRUMENTATION_SCOPE + ".app");
    this.appMeter = appMeter;
    this.battlesTotal = createCounter(appMeter);

    Meter memoryMeter = sdk.getMeter(INSTRUMENTATION_SCOPE + ".memory");
    this.memoryMeter = memoryMeter;
    this.cpuTotal = createGauge(memoryMeter);
}

```

The creation of these two metrics is handled by static methods:

```

static LongCounter createCounter(Meter meter) {
    return meter
        .counterBuilder("battles.total")
        .setDescription("Counts total battles fought.")
        .build();
}

static ObservableDoubleGauge createGauge(Meter meter) {
    return meter
        .gaugeBuilder("jvm.memory.total")
        .setDescription("Reports JVM memory usage.")
        .setUnit("By")
        .buildWithCallback(
            result -> result.record(Runtime.getRuntime().totalMemory(),
                Attributes.empty()));
}

```

The counter is very similar to the Micrometer counter we saw earlier—and when a new battle is fought, we simply increment it, like this:

```
battlesTotal.add(1) .
```

However, the gauge is a little different.

As we can see, the method that actually creates the gauge—`buildWithCallback()`—takes a callback function, which is used to

record the value of the gauge. This will be called only when the gauge is observed—and the order of execution of callbacks between multiple gauges is not specified.

With this branch, the metrics are sent to the OTel Collector but only from the `animal_service`, as this makes things less noisy in the logs, and makes for a better illustrative example.

Only the `debug` exporter is configured for the collector's metrics pipeline, to avoid the need to configure a metrics backend.

OTel metrics also supports JVM-level metrics, and these can be gathered from JMX. There is also support for JFR integration.

It is possible to use the OTel metrics API directly, but many teams prefer the convenience and flexibility of a facading approach (such as Micrometer)—and that's our next topic.

On the branch `micrometer_with_otel`, we show an example of using Micrometer with an OTel exporter. This branch has a dependency on the `micrometer-registry-otlp` library, which provides an OTel exporter for Micrometer:

```
<dependency>
  <groupId>io.micrometer</groupId>
  <artifactId>micrometer-registry-otlp</artifactId>
  <scope>runtime</scope>
</dependency>
```

However, this is provided by the Micrometer libraries, so we do not require the OTel BOM, and there is no direct coupling to OTel in the POM. In fact, the code for `AnimalController` on this branch is identical to the code on the `micrometer_only` branch.

Note that the Micrometer registry is configured in `application.properties`:

```
management.otlp.metrics.export.url=http://otel-collector:4318/v1/metrics
management.otlp.metrics.export.step=10s
```

At the time of writing, this registry supports only compressed HTTP, not gRPC, so we must ensure the Collector is configured to accept OTLP over HTTP by adding the following to the ports section of `docker-compose.yml`:

– "4318:4318" # OTLP http receiver

This exposes the HTTP port as well as the gRPC port, which matches the setting provided in `application.properties`. The configuration presented so far is somewhat bare-bones, but it does provide a starting point for good solutions for metrics and traces. However, we haven't said anything about how to get logs from the application into OTel, so let's discuss that next.

## OpenTelemetry Logs in Java

The development of logs support in OpenTelemetry has proceeded with an eye on the architectural point that logs are expecting to follow the facading patterns already familiar to Java developers.

Therefore, our treatment of logs is going to be somewhat different from that of traces and metrics (where we displayed the low-level “raw” OTel API before discussing alternative approaches).

To be clear, OpenTelemetry does provide a “Logs Bridge” API, which allows logs to be emitted into the OTel pipeline. However, most teams should not use this approach, as it requires too much change to existing practice and codebases. Instead, one of the following alternatives is much more likely to be a better fit for your use case:

- Write logs from your service to a file, using a file-based appender, and arrange for this file to be scraped by the OpenTelemetry Collector, which then forwards to your logging backend via OTLP.
- Use an OTel instrumentation library to export logs from your logging framework of choice to the collector (usually alongside traces and metrics).<sup>5</sup>

There are pros and cons to both approaches—the first seems to serve architectural neutrality, but in practice, proves to be inflexible, whereas the second requires more upfront effort but seems to require less effort to maintain in the medium term.

In what follows, we will focus on the second option—the [OTel logging docs](#) should be consulted if you want to explore the first architectural option.

Unlike in the case of agent-based tracing or Micrometer-based metrics, there is no SPI or facading API for logs that does not involve an explicit dependency on OTel.

We have no choice but to directly couple to the OTEL libraries in our POM. Just as for metrics, we will need to make an `OpenTelemetry` bean available for autowiring. We will also need to configure the Collector to receive logs, but this is a minor detail.

The boilerplate looks like this:

```
@ConditionalOnClass(LoggerContext.class)
@ConditionalOnProperty(name="otel.instrumentation.logback.enabled",
                        matchIfMissing=true)

@Configuration
static class LogbackAppenderConfig {
    @Bean
    ApplicationListener<ApplicationReadyEvent> logbackOtelAppenderInitialize(
        OpenTelemetry openTelemetry) {
        return event -> OpenTelemetryAppender.install(openTelemetry);
    }
}
```

We require a very similar collector config to previously; details can be found on the `logging_only` branch.

We're using the Logback appender, which needs to be included in the POM:

```
<dependency>
  <groupId>io.opentelemetry.instrumentation</groupId>
  <artifactId>opentelemetry-logback-appender-1.0</artifactId>
  <version>2.0.0-alpha</version>
</dependency>
```

and we configure it in `logback.xml` like this:

```
<appender name="OpenTelemetry"
          class="io.opentelemetry.instrumentation.logback.appender.v1_0.
OpenTelemetryAppender">
  <captureExperimentalAttributes>true</captureExperimentalAttributes>
  <captureKeyValuePairAttributes>true</captureKeyValuePairAttributes>
</appender>
```

and include an entry in `application.properties`:

```
otel.instrumentation.logback.enabled=true
```

to activate the conditional beans in the `AnimalApplication` class.

With this setup, developers can just continue to use SLF4J and Logback as normal, and the logs will be sent to the OTel Collector, which will forward them to the logging backend.

Bringing together the different approaches for traces, metrics, and logs, we can see that the recommended stack for Java that our applications could use might look something like:

- OTel agent for traces and JVM metrics
- Micrometer with OTLP registry for application metrics
- SLF4J with Logback OTLP logging appender for logs

The flows for each of these signals could be something like:

- Traces: Java agent → OTLP Exporter → OTel Collector → Jaeger
- Metrics: Micrometer → OTLP Exporter → OTel Collector → Prometheus
- Logs: SLF4J/Logback → OTLP Exporter → OTel Collector → Loki

The use of a local OTel Collector—generally one per cluster—provides a useful indirection point. This makes changing the architecture and handling cases like multipublication much easier, and it also isolates developers from many of the details of exactly how observability is architected.

Of course, this is not the only architectural approach—there are a large number of possible combinations of components that can be used to build an observability system. For example, OTel tracing can be either manual or automatic; Prometheus can be configured to either scrape from the OTel Collector or receive remote write from it.

Ultimately, it is a matter of understanding the overall architecture of the system as a whole—both in its current state and how it might evolve in the future. From this starting point, the team can then make architectural and deployment choices that work well in their particular circumstances.

## Summary

In this chapter, we have taken a deep dive into the practicalities of implementing observability in Java cloud applications.

We have introduced some of the most important technologies in this space (Micrometer, Prometheus, and OpenTelemetry) and shown how they can be used together to build a complete open source observability system. We have also discussed some of the architectural principles that inform decisions about implementing observability systems for Java applications.

Where possible, we have tried to showcase approaches that do not impact the existing working practices of the development team. For example, use of the OpenTelemetry agent to provide automatic tracing, and use of the Micrometer registry for OTel to provide metrics.

This chapter also included a look at components such as the OpenTelemetry Collector and the use of the OTLP protocol for observability data transport.

One notable omission from this chapter is the subject of application profiling. This is a very important topic, but it is also a very large one, and it does not fit neatly into the framework for observability we have just introduced. In fact, it warrants a separate chapter of its own, and that is where we will turn our attention next.

- <sup>1</sup> Status of the OTel components for various major languages can be found in [OTel's documentation](#).
- <sup>2</sup> These promises might be weaker than reality, specifically for [opentelemetry-java](#). Supporting and providing backward compatibility on stable components is a major goal for the community, and publishing a major version would be an indication that breaking changes were being made.
- <sup>3</sup> See also the [semantic conventions](#).
- <sup>4</sup> Other encodings are possible but not often seen in practice.
- <sup>5</sup> Technically, if you use this option, you are using the Logs Bridge, just not directly.